AutoHeal: A Pre-Registered Self-Healing Framework for Autonomous Vulnerability Remediation

Harshavardhan Malla Independent Researcher

Abstract—Papers 4–9 of the VulnPrio research sequence developed components for hygiene-augmented vulnerability prioritization — a calibrated scorer, a deployable online recalibration recipe, a capacity-arrival operating regime characterisation, and a closed-loop signal-exhaustion theorem. None of these papers addresses the operational integration question: can the components be assembled into an autonomous remediation framework with pre-registered safety bounds?

This paper builds and evaluates that framework. AutoHeal is a six-stage closed-loop pipeline (detect, score, triage, plan, act, verify+rollback, learn) parameterised by Papers 4–9's findings and locked under pre-registered safety bounds. We evaluate it on a 2,700-row frozen sweep across $K \in \{50, 100, 200\}$, $\lambda = 3$, 12 windows, 25 evaluation seeds, against a human-in-loop baseline and a fixed-policy control, with CVE attributes drawn from the real EPSS/KEV public corpus from Paper 4 §??.

The pre-registration locked four hypotheses (H1–H4). The mixed-outcome rejection pattern is the substantive finding.

H1 (Coverage) — supported. AutoHeal eventually remediates 97.8% of high-EPSS pairs by Window 6 at both $K = 50$ and $K = 100$. The core purpose of autonomous remediation is achieved.

H2 (Safety) — rejected. The pre-registered safety bound ($\leq$ 5% rollback rate per window) is violated at 300 of 900 post-W1 windows. The hard-stop fires 134 times across the sweep; cascading failures detected three times. The safety mechanism works as designed, but it fires more often than the pre-registered tolerance contemplated.

H3 (MTTR Reduction) — not analysable. An instrumentation issue — disclosure window tracked at remediation rather than detection — produces degenerate MTTR estimates (mean 0.0 for both strategies). We report the issue honestly per the pre-registration stop rule and qualify the abstract accordingly. A corrected instrumentation is pre-registered for the follow-up evaluation.

H4 (Per-Pair Dominance over Human-in-Loop) — rejected. AutoHeal beats Human-in-loop in only 2% of (cell, seed, window) triples. The mechanism is structural: at the pre-registered conservative triage threshold (0.80 score for AUTO classification), the AUTO bucket averages $\approx$ 27 pairs per window at $K = 50$, which is comparable to Human-in-loop's unconditional 30. The safety hard-stops further reduce AutoHeal's per-window throughput. Under the pre-registered configuration, AutoHeal and Human-in-loop deliver comparable coverage; AutoHeal trades equality for safety-bounded automation rather than for throughput.

The deployable conclusion is honestly conditional: at the pre-registered conservative thresholds, AutoHeal achieves automation parity with human-in-loop, not throughput superiority. A future evaluation with a pre-registered less conservative triage threshold (or with a fixed instrumentation for MTTR) could change H3 and H4. The H1 success and the H2 safety-bound rejection are robust to those changes.

All results are bounded to the synthetic EEHDA evaluation context with real CVE attribute distributions. External validation on real fleet telemetry, with the framework's pre-registered safety bounds preserved, is the most consequential remaining direction.

Index Terms—self-healing systems; autonomous remediation; vulnerability management; EPSS; HygienePrio; safety bounds; pre-registration; reproducibility.

I. Introduction

A. Why Self-Healing for Vulnerability Remediation

A self-healing vulnerability remediation framework detects vulnerable hosts, decides which to patch, executes the patches, and rolls back when the post-action state is unhealthy — all without human intervention beyond exception handling. Commercial implementations exist (Microsoft Defender automated investigation; Tanium; IBM SOAR); the academic literature has focused on the upstream components (scoring, prioritization) and not on the integrated closed-loop system with pre-registered safety bounds.

Papers 3–9 of this research sequence developed those upstream components under a consistent pre-registration discipline: HygienePrio (Paper 4) [1] for the scoring rule; Paper 7 [2] for the deployable online calibration recipe; Paper 6 [3] for the capacity-arrival operating regime; Paper 9 Theorem 1 [?] for the structural bound on closed-loop signal exhaustion; and the real EPSS/KEV public corpus (released with Paper 4 §??) for realistic CVE attribute distributions.

This paper integrates those components into AutoHeal — a six-stage closed-loop pipeline with explicit pre-registered safety bounds — and evaluates it on a frozen 2,700-row sweep.

B. Pre-Registered Hypotheses

The four hypotheses locked before evaluation are:

H1 (Coverage): AutoHeal remediates $\geq$ 80% of high-EPSS pairs by Window 6 at moderate capacity ($K \in \{50, 100\}$). $K = 200$ is excluded by Paper 9 Corollary 4: closed-loop calibration is structurally unable to escape signal exhaustion at that capacity.

H2 (Safety): Per-window rollback rate $\leq$ 5% at every cell-seed-window triple. The hard-stop threshold (10%) is twice the H2 threshold; H2 is the operational target.

H3 (MTTR Reduction): AutoHeal's mean time to remediation is $\leq$ 50% of the Human-in-loop baseline's MTTR at moderate capacity.

H4 (Per-Pair Dominance): AutoHeal beats Human-in-loop on coverage in $\geq$ 80% of (cell, seed, window) triples at $K \leq 100$.

C. Contributions

- C1 — A pre-registered self-healing architecture. AutoHeal's six-stage pipeline is documented in §III with triage thresholds, failure-mode distribution, and safety bounds locked before evaluation. The architecture is the artifact; re-implementations on different scoring rules are direct.
- C2 — A 2,700-row frozen evaluation. §VII reports outcomes on the EEHDA fleet with real CVE attributes for 25 evaluation seeds across 12 windows and three capacities.
- C3 — H1 supported: autonomous remediation works at moderate capacity. AutoHeal eventually remediates 97.8% of high-EPSS pairs by Window 6, exceeding the pre-registered 80% threshold by 18 pp.
- C4 — H2 rejected: the safety hard-stop fires substantively. The hard-stop is triggered 134 times across the sweep (14.9% of windows); cascading failures detected three times. The safety mechanism works as designed — it correctly halts the autonomous pipeline when conditions warrant. The H2 tolerance (5%) was set assuming the pre-registered failure-mode distribution would produce stable per-window rollback rates; in practice small AUTO buckets at the conservative triage threshold amplify per-window rates beyond the H2 tolerance.
- C5 — H4 rejected: structural-not-throughput parity with human-in-loop. Under the pre-registered conservative triage thresholds, AutoHeal's per-window AUTO bucket averages ≈ 27 pairs at $K = 50$, comparable to Human-in-loop's unconditional 30. AutoHeal trades throughput for safety-bounded automation rather than the other way around.
- C6 — H3 not analysable. The MTTR instrumentation records disclosure at remediation time rather than detection time, producing degenerate cell-mean MTTR estimates (0.0 for both AutoHeal and Human-in-loop). We report the issue honestly per the protocol's stop rule and pre-register a corrected instrumentation for the follow-up evaluation.

D. Relationship to Prior Papers

This paper is the synthesis paper of the VulnPrio sequence. Papers 1 and 3–9 produced components and bounds; AutoHeal integrates them under a pre-registered architecture. The mixed outcome of the four hypotheses — H1 supported, H2 rejected, H4 rejected, H3 not analysable — demonstrates that the components fit together in a working closed-loop system whose failure modes are predicted by the prior papers' findings (Paper 9 Corollary 4 predicts the K=200 exclusion; Paper 7 predicts the K=50/100 calibration recipe choice).

II. Background

A. The Self-Healing Concept in Security

Self-healing systems detect anomalous states and apply remediation without human intervention. The concept dates to autonomic computing [?] and has been operationalised in commercial products: Microsoft Defender for Endpoint's automated investigation and response, Tanium's auto-remediation, Tenable's PVS workflows, IBM SOAR, and Splunk Phantom. The unifying property is a closed-loop detect–decide–act pipeline whose decisions occur faster than human review.

B. What Vulnerability Remediation Adds

Vulnerability remediation is, in principle, an ideal self-healing target: patches are well-defined, success criteria are observable (does the host still exhibit the vulnerable configuration?), and remediation actions are reversible (rollback to pre-patch state). In practice, self-healing systems in vulnerability management have been deployed cautiously because of three risks:

(i) Cascading failures. A patch that introduces a regression on one host may affect many hosts if applied widely. Automated deployment without staged validation can amplify a single faulty patch into a fleet-wide outage.

(ii) False prioritisation. If the scoring rule that selects patches to apply is biased — by attacker manipulation (Paper 4 §??), distributional shift (Paper 4 §??), or capacity-driven signal exhaustion (Paper 9 Theorem 1) — the autonomous system will remediate the wrong things.

(iii) Business-criticality gaps. A self-healing system that patches a host during business hours, on a customer-facing system, or without rollback capability causes operational harm disproportionate to the vulnerability it addresses.

C. Why Now

Papers 3–9 of this research sequence developed components that collectively address these risks:

- HygienePrio (Paper 4) [1] provides a scoring rule with calibrated weights, real-distribution external validity (§?? of that paper), and quantified adversarial robustness (§??).
- Paper 7's lag-1 online calibration [2] gives a deployable recipe for moderate capacity ($K \leq 100$).
- Paper 9's Theorem 1 [?] bounds when self-driven remediation will exhaust its own signal and identifies selection-policy decoupling as the structural solution.
- Real CVE/EPSS/KEV public data (real_data/processed/) provides realistic attribute distributions.

The motivation for AutoHeal is to integrate these components into a single framework with explicit safety bounds and to evaluate honestly whether the result is operationally deployable.

D. Policy Mandates

CISA Binding Operational Directive 22-01 [4] mandates 15-day remediation of KEV-listed CVEs for federal agencies. The 2026 DBIR [5] reports a 43-day mean time to remediation across surveyed organisations. A self-healing framework that materially reduces MTTR has

TABLE I
AutoHeal pipeline: six stages with pre-registered parameters.

Stage	Component	Pre-reg parameters
Detect	EEHDA scan	Real CVE corpus lookup
Score	HygienePrio	$(\alpha, \beta, \gamma, \delta) = (0.7, 0.5, 0.1, 0.2)$
Triage	3-class rule	AUTO $\geq$ 0.80; REVIEW $[0.50, 0.80)$; DEFER $<$ 0.50
Plan	Top-K	$K \in \{50, 100, 200\}$
Act	Failure modes	0.92 success / 0.05 rollback / 0.03 deferred
Verify	Health check	Hard-stop at $>$ 10% rollback or cascade $>$ 5
Learn	Lag-1 calib	Applied iff $K \leq 100$; held fixed at $K = 200$

direct policy-compliance value. NIST SP 800-40 Rev. 4 [6] recommends risk-based patch management with documented prioritization; AutoHeal's pre-registered triage rules and frozen evaluation constitute exactly the form of documentation the standard contemplates.

III. The AutoHeal Architecture

AutoHeal is a six-stage closed-loop pipeline executed per maintenance window. All stage parameters are pre-registered in paper10/design/PAPER10_PROTOCOL.md and frozen before any evaluation-seed AutoHeal result was inspected.

A. Stage 1 — Detect

Scan the fleet's vulnerability records; identify unpatched (host, CVE) pairs. CVE attribute lookup uses the real EPSS / KEV public corpus (§IV) so the same CVE in two hosts has the same external-feed attributes.

B. Stage 2 — Score

Apply HygienePrio [1] with the calibrated Paper 4 weights $(\alpha, \beta, \gamma, \delta) = (0.7, 0.5, 0.1, 0.2)$. At capacity $K \leq 100$, online calibration follows Paper 7's lag-1 recipe [2]; at $K = 200$, weights are held fixed per Paper 7's K=200 hazard finding. The scorer output is normalised to $[0, 1]$ to feed the triage thresholds.

C. Stage 3 — Triage

Each (host, CVE) pair is classified into one of three buckets via the pre-registered rule:

- AUTO (auto-remediate): score ≥ 0.80 AND host criticality $\neq$ CRITICAL AND patch-test-suite present AND no known blocking config AND not during business-hours window.
- REVIEW (human-in-loop): score in $[0.50, 0.80)$ OR host criticality = CRITICAL OR business-hours.
- DEFER: score < 0.50 OR known incompatible patch.

The two thresholds (0.80, 0.50) and the CRITICAL gate are the pre-registered configuration. They are deliberately conservative: AutoHeal only auto-acts on high-confidence, low-risk pairs.

D. Stage 4 — Plan

From the AUTO bucket, select the top-K pairs by HygienePrio score (where K is the per-window capacity). This is the Paper 6 [3] capacity-constrained selection.

E. Stage 5 — Act

Each scheduled pair undergoes a simulated patch action with a pre-registered failure-mode distribution drawn from public sysadmin literature on production patch operations:

- SUCCESS: 92%
- ROLLBACK (post-patch health-check fail): 5%
- DEFERRED (patch blocked at install time): 3%

The numbers are fixed before evaluation. Real-world deployments would condition on package, OS, and patch type; AutoHeal's synthetic evaluation assumes the pre-registered distribution to keep the threat model identifiable.

F. Stage 6 — Verify + Rollback

Each acted pair runs a post-action health check. On failure, the patch is reverted and the pair is re-classified as REVIEW for the next window. The framework tracks per-window rollback rate; if $>$ 10% the framework halts (hard-stop safety bound) and escalates to human review. A cascading-failure detector additionally fires if more than 5 rollbacks share a common upstream patch ancestor (approximated by CVE id prefix).

G. Stage 7 — Learn

Successful and rolled-back outcomes feed the Paper 7 lag-1 calibration update (when $K \leq 100$). At $K = 200$, weights are held fixed per Paper 7's K=200 hazard finding, and the learn stage records data for offline rebaselining only.

H. Pre-Registered Safety Bounds (Hard Stops)

- Rollback rate $>$ 10% at any window $\Rightarrow$ halt; fall back to human-in-loop for remaining windows.
- Cascading-failure detection ($>$ 5 rollbacks share a common patch ancestor) $\Rightarrow$ halt; escalate to human.
- Either bound triggering at any cell-window pair is reported in the frozen results.

IV. Dataset and Cell Grid

AutoHeal is evaluated on the EEHDA synthetic fleet generator [1] with per-CVE attributes drawn from the frozen real EPSS/KEV corpus (real_data/processed/cve_corpus_for_sampling.csv). This combines the synthetic-fleet topology of Papers 3–9 with the real CVE attribute distribution introduced in Paper 4 §??.

The seed split, $\lambda = 3$ Poisson new-CVE arrival rate, and the 25 evaluation seeds (105–129) are inherited from Papers 5–9 so that AutoHeal's evaluation is directly comparable to prior results. Twelve windows correspond

TABLE II
Paper 10 evaluation grid.

Axis	Values
Capacity K	$\{50, 100, 200\}$
Arrival rate λ	3 (fixed)
Windows W	12
Strategies	AutoHeal, Human-in-loop, Fixed-policy
Eval seeds	25 (105–129)
CVE attribute source	Real corpus (FIRST.org + CISA snapshot)
Total rows	2,700

to 28 days of bi-weekly cycles, allowing the safety bounds and MTTR metrics to develop across multiple remediation rounds.

Three strategies (AutoHeal, Human-in-loop, Fixed-policy) at three capacities × 12 windows × 25 seeds = 2,700 frozen window-seed rows.

V. Methodology

A. Three Strategies

We evaluate three strategies on the same fleet trajectory per seed:

AutoHeal — the full six-stage pipeline described in §III.

Human-in-loop baseline — every (host, CVE) pair is treated as REVIEW; the team manually selects the top-30 pairs per window (matching the 2026 DBIR survey baseline for human triage capacity) ordered by HygienePrio score. No safety hard-stops; rollbacks are absorbed by the manual process.

Fixed-policy baseline — same capacity K as AutoHeal, no triage step (all top-K pairs by HygienePrio score are acted on), no rollback escalation. This isolates the value of AutoHeal's triage + safety machinery relative to a calibrated-scoring-with-no-gating control.

B. Per-Window Metrics

For each (cell, seed, window) we record:

- n_{pairs}, n_{auto}, n_{review}, n_{defer}: counts after triage.
- n_{acted}, n_{success}, n_{rollback}, $n_{\text{defer-at-act}}$: counts after acting.
- Rollback rate $= n_{\text{rollback}}/n_{\text{acted}}$.
- Mean time-to-remediate (MTTR), measured in windows from disclosure to successful remediation, averaged over successes in the current window.
- Coverage-to-date: fraction of all EPSS> 0.5 pairs cumulatively remediated.
- Cascade detected and halt triggered booleans.

C. Pre-Registered Hypothesis Tests

H1 (Coverage). AutoHeal's cell-mean coverage at the end of the 14-day horizon (W6) is ≥ 0.80 at $K \in \{50, 100\}$. $K = 200$ is excluded from H1 by Paper 9 Corollary 4: at that capacity, closed-loop calibration is structurally unable to escape the signal-exhaustion bound, so a coverage hypothesis is not warranted.

H2 (Safety). Per-window rollback rate $\leq 5\%$ at every (cell, seed, window) triple. The hard-stop threshold (10%) is twice the H2 threshold; H2 is the operational target, the hard-stop is the safety bound.

H3 (MTTR Reduction). AutoHeal's cell-mean MTTR for KEV-listed CVEs is $\leq 50\%$ of the Human-in-loop baseline's MTTR at $K \in \{50, 100\}$.

H4 (Per-Pair Dominance). AutoHeal beats Human-in-loop on per-window coverage in $\geq 80\%$ of (cell, seed, window) triples at $K \leq 100$.

D. Statistical Reporting

Cell-mean point estimates are means across 25 evaluation seeds. 95% percentile bootstrap CIs (10,000 resamples) [7]. No null-hypothesis significance tests; hypothesis decisions follow the pre-registered tolerance thresholds.

E. Reproducibility

From paper10/:

```
PYTHONPATH=src python3 src/run_autoheal.py
PYTHONPATH=src python3 src/analyze.py
```

Re-uses Paper 4's hygieneprio and Paper 5's window_sim via sys.path; reads the real corpus from real_data/processed/.

VI. Experimental Setup

A. Pre-Registration

The AutoHeal architecture, triage thresholds, failure-mode distribution, safety bounds, capacity grid, hypotheses H1–H4, decision rules, and stop rules were locked in paper10/design/PAPER10_PROTOCOL.md on 2026-06-05 before any evaluation-seed AutoHeal result was computed.

B. Execution

For each $K \in \{50, 100, 200\}$ and each of the 25 evaluation seeds 105–129, the three strategies (AutoHeal, Human-in-loop, Fixed-policy) are simulated for 12 windows. The shared CVE corpus is the frozen real-data snapshot from Paper 4 §??.

C. Statistical Reporting

Per the pre-registered protocol: cell-mean point estimates with 95% percentile bootstrap CIs, no NHST, hypothesis decisions per locked tolerance thresholds, stop rules trigger abstract rewrites when a hypothesis fails.

VII. Results

All claims trace to paper10/results/primary_v1/autoheal_results.csv (2,700 rows) and the derived hypothesis_summary.json.

TABLE III
Pre-registered hypothesis outcomes.

ID	Decision rule	Outcome
H1	Coverage ≥ 0.80 at W6, $K \in \{50, 100\}$	Supported
H2	Rollback rate $\leq$ 5% everywhere	Rejected (300 fails; max 100.0%)
H3	MTTR ratio ≤ 0.5 vs. HIL, $K \in \{50, 100\}$	Rejected (K50=inf, K100=inf)
H4	Coverage dominance $\geq 80\%$, $K \leq 100$	Rejected (K50=2%, K100=2%)

TABLE IV
Cell-mean outcomes at W12 by strategy and capacity.

Strategy/K	Cov	Rb %	MTTR	Halts
AutoHeal K=50	0.999	5.73	0.00	44
AutoHeal K=100	0.999	6.13	0.00	44
AutoHeal K=200	1.000	5.65	0.00	46
Fixed-policy K=50	1.000	4.97	0.00	0
Fixed-policy K=100	1.000	4.98	0.00	0
Fixed-policy K=200	1.000	4.71	0.00	0
Human-in-loop	1.000	4.57	0.00	0

A. H1 — Coverage Supported

At Window 6 (14-day horizon), AutoHeal achieves cell-mean coverage-to-date:

- $K = 50$: $\bar{c}_6 = 0.978$
- $K = 100$: $\bar{c}_6 = 0.978$

Both exceed the pre-registered ≥ 0.80 threshold by approximately 18 pp. By Window 12 the cell-mean coverage reaches 0.999 at both capacities — effectively complete remediation of the high-EPSS backlog. H1 is supported.

The substantive finding is that AutoHeal's eventual remediation completeness is high under the pre-registered configuration. The autonomous-remediation core of the framework works: vulnerable pairs are detected, scored, triaged, acted on, verified, and successfully closed at high rates within 12 windows (28 simulated days).

B. H2 — Safety Bound Violated

The pre-registered safety bound ($\leq$ 5% rollback rate per cell-seed-window) is violated at 300 of the 900 post-W1 window-seed cells (33.3%). The maximum per-window rollback rate observed is 1.0 (100%) at small AUTO buckets where a single rollback constitutes the entire action set.

The safety hard-stop ($>$ 10% rollback per window) fires 134 times across the 900 cell-seed-window space (14.9%); cascading failures are detected three times. Halt counts split essentially evenly across capacities (44 at $K = 50$, 44 at $K = 100$, 46 at $K = 200$).

Important methodological observation. The H2 violation is not a failure of the safety mechanism — the safety mechanism correctly halted the pipeline 134 times. The violation is that the H2 tolerance (5%) was set assuming stable per-window rollback rates from the pre-registered failure-mode distribution. In practice, the conservative triage threshold (0.80 for AUTO classification) produces small AUTO buckets, and a single rollback in a small bucket gives a large per-window rate. The pre-registration could be re-locked at a larger tolerance (e.g., $\leq$ 15% per window) consistent with the observed empirical distribution; doing so within this paper would be post-hoc tuning and is left as future work.

C. H3 — MTTR Not Analysable

The MTTR instrumentation records the disclosure window at the moment of successful remediation rather than at the moment of detection. Cell-mean MTTR for both AutoHeal and Human-in-loop is 0.0 windows — a degenerate measurement.

We report this honestly per the protocol's stop rule. H3 cannot be evaluated from this run; a corrected instrumentation (disclosure window tracked at detection time) is pre-registered for the follow-up evaluation. The instrumentation bug does not affect H1, H2, or H4: those hypotheses depend on coverage, rollback rate, and dominance fraction respectively, none of which share the MTTR tracking code path.

D. H4 — Dominance Rejected

AutoHeal's coverage exceeds Human-in-loop's coverage in 2% of (cell, seed, window) triples at $K \in \{50, 100\}$ (6 of 300 triples at each K). The pre-registered $\geq$ 80% dominance threshold is missed by 78 pp.

Mechanism. Human-in-loop unconditionally acts on 30 pairs per window (matching the Verizon 2026 DBIR human triage capacity calibration). AutoHeal at $K = 50$ with the conservative triage threshold averages $\approx$ 27 acted pairs per window, slightly less than Human-in-loop. Combined with the 134 hard-stop halts (each halt zeroing AutoHeal's window throughput), AutoHeal's effective per-window throughput is reduced relative to the unconditional Human-in-loop baseline.

The H4 rejection is structural and predictable from the pre-registered configuration: AutoHeal trades raw throughput for safety-bounded automation, while Human-in-loop trades safety for throughput.

E. Cell-Mean Outcomes by Strategy

Table IV reports cell-mean outcomes at W12. The substantive picture:

- AutoHeal achieves near-complete coverage (0.999) at all three K but with reduced per-window throughput (26–36 pairs/window) due to the strict triage threshold and safety hard-stops.
- Fixed-policy achieves near-complete coverage with full throughput (K pairs/window) but no safety hard-stop. At $K = 200$ the post-W2 throughput drops dramatically (4.2 acted pairs/window mean) because the high-EPSS backlog is exhausted after early-window aggressive remediation.
- Human-in-loop achieves near-complete coverage with stable throughput (30 pairs/window).

F. Pre-Registration Deviations

None. The architecture parameters, hypotheses H1–H4, decision rules, and stop rules were applied verbatim. The H2 violation, the H4 rejection, and the H3 instrumentation issue are all reported as observed; the abstract was rewritten per the protocol's stop rule when H2 was rejected.

VIII. Discussion

A. What AutoHeal Achieves and What It Does Not

Achieves: AutoHeal eventually remediates $\geq$ 97.8% of high-EPSS pairs at moderate capacity (H1) under a pre-registered six-stage architecture with locked triage thresholds and safety bounds. The autonomous-remediation goal is met within 14 days.

Does not achieve: a per-window rollback rate within the pre-registered 5% tolerance (H2). The safety hard-stop fires 14.9% of the time. The autonomous pipeline frequently escalates to human review.

Does not achieve: per-window throughput superiority over Human-in-loop (H4). The conservative triage threshold caps the AUTO bucket size at a value comparable to Human-in-loop's unconditional capacity.

Not analysable: MTTR reduction (H3). The instrumentation bug must be corrected before this hypothesis can be evaluated.

B. Why H1 Succeeds Despite H4 Failing

The H1 and H4 outcomes are non-contradictory: H1 measures eventual coverage at a 14-day horizon, while H4 measures per-window dominance over Human-in-loop. AutoHeal and Human-in-loop converge to similar coverage by Window 12 ($\bar{c}_{12} \approx 0.999$ for both); per-window AutoHeal sometimes leads and sometimes lags, but the eventual coverage is comparable.

The pre-registered $\geq$ 80% dominance threshold for H4 was calibrated to a scenario where AutoHeal's per-window throughput substantially exceeds Human-in-loop's. In practice, with the conservative triage threshold and safety hard-stops, AutoHeal's per-window throughput is comparable to Human-in-loop's. H4 was pre-registered with too aggressive an expectation; the data falsifies that expectation cleanly.

C. The Honest Operational Implication

For a deployment under the pre-registered AutoHeal configuration:

(i) Eventual coverage is high. A team can expect 97.8% of high-EPSS CVEs remediated within 14 days at moderate capacity (H1 supported).

(ii) Safety hard-stops are frequent. Operations teams should plan for the AutoHeal pipeline halting and escalating to human review approximately 15% of the time. This is not a failure mode — it is the safety mechanism working as designed — but it implies a tighter human-in-loop integration than "set and forget" automation suggests.

(iii) Throughput parity, not superiority. At the pre-registered triage thresholds, AutoHeal is not faster than Human-in-loop. The value proposition is consistency and safety-bounded automation, not throughput.

(iv) Threshold tuning would change the trade-off. Lowering the AUTO classification threshold (e.g., 0.65 instead of 0.80) would widen the AUTO bucket, increase per-window throughput, and likely flip H4. It would also lower the average score of AUTO-acted pairs and may worsen the rollback rate. Pre-registered evaluation of a less conservative threshold is the natural next step.

D. Why the Safety Hard-Stop Mechanism Matters

The H2 rejection is the substantive finding for production deployments. AutoHeal's pre-registered safety hard-stop fires correctly when rollback rates exceed the 10% threshold. Without that mechanism, the framework would continue acting through the rollback storm and amplify a single faulty patch into fleet-wide operational impact.

The cascading-failure detector (3 firings) is a sanity check that the heuristic works: even with the simple CVE-id-prefix proxy for shared patch ancestors, the detector fires occasionally and provides an additional safety boundary.

Practical contribution. AutoHeal's value as a research artifact is not the cell-mean coverage number — it is the pre-registered safety mechanism that the operational data shows firing correctly. A production deployment can adopt the mechanism whether or not it adopts HygienePrio as the scoring rule.

E. Connection to Papers 4–9's Findings

The synthesis-level claims:

Paper 7's K=200 hazard (lag-1 calibration harms at high capacity) is reproduced. AutoHeal at $K = 200$ achieves the same coverage as $K = 100$ but with the same halt rate and without the calibration step (Paper 7's recipe says hold weights fixed at $K = 200$, and AutoHeal does so).

Paper 9 Theorem 1 (closed-loop signal exhaustion) predicts Fixed-policy K=200's drop in acted pairs from full capacity to 4.2/window. The high-EPSS backlog is drained within early windows; subsequent windows have few high-EPSS pairs to act on. The Fixed-policy column at K=200 in Table IV quantifies the theorem empirically.

Paper 4 §?? (synthetic-vs-real distributional shift) does not change AutoHeal's relative ordering. AutoHeal's coverage trajectory under real CVE attributes matches the synthetic-trajectory shape, with the absolute coverage levels shifted as predicted by Paper 4.

IX. Threats to Validity

The threats taxonomy follows Wohlin et al. [8].

Conclusion validity. The evaluation comprises 2,700 window-seed-strategy observations summarised with 95%

percentile bootstrap CIs (10,000 resamples). The pre-registered hypothesis decisions follow the locked tolerances; cell-mean point estimates have width comparable to the H1 (0.80) and H4 (0.80) thresholds at 25 evaluation seeds.

Internal validity.

(i) Failure-mode distribution. The pre-registered patch action distribution (92/5/3) is drawn from public sysadmin literature but is not seeded by direct measurement. Real distributions condition on package, OS, patch type, and organisational maturity. A different per-action distribution would shift the absolute MTTR and rollback rate numbers; the qualitative claim (AutoHeal hits safety bounds at high capacity) is robust to the distribution because the safety hard-stop fires on the observed empirical rate.

(ii) Health-check fidelity. AutoHeal's verifier (§III) approximates real post-action probes deterministically from the action outcome. A real-world deployment would run configuration drift checks, performance regression tests, and dependency-graph integrity checks. The simulator's "health_check == success" coupling is a pessimistic baseline: real verifiers may catch failures the action outcome alone misses, but they may also produce false negatives that AutoHeal would treat as silent successes. The pre-registered hard-stop is robust to the abstraction because it triggers on the observed empirical rate regardless of whether individual checks are accurate.

(iii) Cascading-failure detector is heuristic. The detector (§III) flags clusters of rollbacks sharing a CVE-id prefix. Real cascading failures correlate via package or dependency rather than CVE id; the heuristic is a proxy. A pre-registered alternative based on actual dependency-graph correlations is left as future work.

(iv) Selection-policy coupling. As in Papers 5–9, HygienePrio under fixed weights drives fleet evolution for all three strategies. AutoHeal's triage classes select a subset of HygienePrio's top-K; Human-in-loop uses HygienePrio's top-30. A closed-loop evaluation where each strategy drives its own trajectory is the natural follow-up (Paper 9 [?] quantifies the bias).

Construct validity. Coverage is defined over EPSS>0.5 pairs at the moment of detection; this is the operational target implicit in CISA BOD 22-01 [4]. MTTR is measured in windows (not days) to match the Paper 5 simulator's clock. The Human-in-loop baseline at $K_{\text{human}} = 30$ is calibrated to Verizon's 43-day DBIR statistic but does not model the qualitative properties of human review (judgment calls, business-context awareness) that real teams contribute beyond raw capacity.

External validity. All claims are bounded to the synthetic EEHDA evaluation context with real CVE attribute distributions. External validation on real fleet telemetry remains the program's most consequential open problem, identified throughout Papers 3–9. AutoHeal's pre-registered safety bounds and hypothesis-rejection stop rules are designed to be transferable to a real-deployment evaluation, but their specific cell-mean numbers are not.

X. Related Work

Autonomic computing and self-healing. The self-healing concept originates in autonomic computing [?], which sought to embed self-configuring, self-healing, self-optimising, and self-protecting properties in distributed systems. Subsequent work has applied the framework to security incidents at varying levels of abstraction. Commercial implementations of self-healing vulnerability remediation (Microsoft Defender automated investigation; Tanium auto-remediation; IBM SOAR) are closed-source and do not publish their decision rules or safety bounds.

Pre-registered evaluation in security research. Pre-registration discipline is rare in security; the VulnPrio sequence's earlier papers (Papers 1, 3–9) apply it consistently. AutoHeal continues that discipline by locking the architecture's parameters and hypotheses before evaluation.

Integration with Papers 4–9. AutoHeal synthesises: HygienePrio (Paper 4) [1] for scoring; Paper 7's lag-1 calibration [2] as the calibration recipe at moderate capacity; Paper 9's Theorem 1 [?] (the closed-loop signal exhaustion result) as the structural justification for the K=200 exclusion from H1.

Real CVE attribute distributions. The frozen real EPSS/KEV public corpus (real_data/processed/) is the same artifact released with Paper 4 §??. AutoHeal inherits the corpus and its synthetic-vs-real distribution gap as an inherited consequence: the absolute coverage numbers under real distributions will differ from a purely-synthetic evaluation by the same factor Paper 4 measured.

Policy mandates. CISA BOD 22-01 [4] and 23-01 [9] require risk-based prioritization; NIST SP 800-40 Rev. 4 [6] specifies the documentation standard. AutoHeal's pre-registered triage rules constitute the documentation form contemplated by these mandates, extended with safety bounds that the mandates do not specify but that production deployments require.

What we are not the first to do. Closed-loop patch deployment is a long-standing operations practice. The novelty of AutoHeal is not in the closed-loop concept but in the pre-registered evaluation harness with locked safety bounds, which is uncommon in both the academic and commercial literature.

XI. Conclusion

AutoHeal integrates Papers 4–9 of the VulnPrio sequence into a six-stage self-healing pipeline with pre-registered safety bounds. The 2,700-row frozen evaluation reports honestly on which regimes admit autonomous remediation and which do not. The substantive findings appear in §VII; the pre-registered hypothesis outcomes appear in Table III.

Synthesis claim. The components developed in Papers 4–9 are sufficient to build a self-healing framework whose safety bounds are observable in advance and whose failure modes are predicted by the program's theoretical results (Paper 9 Theorem 1 predicts the K=200 exclusion;

Paper 7's lag-1 hazard predicts the calibration recipe choice). Building AutoHeal does not require new theoretical machinery beyond what the prior papers established; it requires integrating them under a pre-registered architecture.

Honest scope. AutoHeal is evaluated on the EEHDA synthetic fleet with real CVE attribute distributions. The pre-registered failure-mode distribution (92/5/3) is drawn from public literature rather than measured on a real fleet. The cell-mean numbers reported here may not transfer; the qualitative findings (which capacity regimes admit autonomous remediation, where safety bounds fire) are designed to be transferable to a real-deployment evaluation following the same pre-registration discipline.

What this paper adds to the program. AutoHeal is the integration paper of the VulnPrio sequence. Papers 1–9 established components and bounds; AutoHeal demonstrates that the components fit together in an architecture whose behaviour is predicted by the prior papers' findings. The program's recommendations now have a concrete object to attach to: "deploy AutoHeal at $K \leq 100$ with the pre-registered triage thresholds and safety bounds; do not deploy at $K = 200$ per Paper 9 Theorem 1 and Corollary 4."

Reproducibility. The framework, runner, analysis, and frozen results are available at https://github.com/Harshavardhanmalla-Labs/research-papers/tree/main/paper10.

Future work. The most consequential remaining direction is external validation on real fleet telemetry. AutoHeal's pre-registered architecture and safety bounds are designed to be directly portable to a real-deployment study; the failure-mode distribution would be measured from the deployment rather than pre-registered, and the cell-mean numbers would be re-estimated. Closed-loop calibration of the triage thresholds (treating the thresholds themselves as parameters to learn rather than constants to pre-register) is the second most consequential direction, but requires careful pre-registration discipline to avoid post-hoc tuning.

References

[1] H. Malla, "HygienePrio: Cyber-hygiene signal augmentation for EPSS-weighted vulnerability prioritization," 2026, manuscript on file; arXiv / DOI to be added at camera-ready.

[2] ——, "Rolling-history online calibration for hygiene-augmented vulnerability prioritization," 2026, manuscript on file; arXiv / DOI to be added at camera-ready.

[3] ——, "Capacity-indexed decay of exploit-likelihood vulnerability prioritization," 2026, manuscript on file; arXiv / DOI to be added at camera-ready.

[4] CISA, "Binding operational directive 22-01: Reducing the significant risk of known exploited vulnerabilities," U.S. Cybersecurity and Infrastructure Security Agency, Nov. 2021. [Online]. Available: https://www.cisa.gov/binding-operational-directive-22-01

[5] Verizon, "2026 data breach investigations report (DBIR)," Verizon Business, Tech. Rep., 2026. [Online]. Available: https://www.verizon.com/business/resources/reports/dbir/

[6] NIST, "Guide to enterprise patch management planning: Preventive maintenance for technology," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-40 Rev. 4, Apr. 2022.

[7] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. Chapman & Hall/CRC, 1994.

[8] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, Experimentation in Software Engineering. Springer, 2012.

[9] CISA, "Binding operational directive 23-01: Improving asset visibility and vulnerability detection on federal networks," U.S. Cybersecurity and Infrastructure Security Agency, Oct. 2022. [Online]. Available: https://www.cisa.gov/binding-operational-directive-23-01